

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

ADVANCED COMPUTER ARCHITECTURE

Lab 5

Thibaut Beck
Wannes Paesschesoone

Academic year 2025–2026

Contents

1 Introduction **2**

2 Results **2**

2.1 Discussion 2

3 Code **3**

1 Introduction

In GPU programming, there are two main ways to execute functions: synchronous and asynchronous calls. A synchronous function call means that the CPU waits until the GPU has completely finished its task before continuing with the next instruction. This creates a clear execution order, but it also means that the CPU remains idle while the GPU is working.
In contrast, an asynchronous function call allows the CPU to continue executing other instructions while the GPU is still processing. The tasks are launched on the GPU, and the CPU does not wait for their completion. This can lead to better overall performance, especially when CPU and GPU operations can overlap efficiently. However, it also introduces more complexity in synchronization and memory management.
In this lab, we study the difference between synchronous and asynchronous function calls on the GPU. Each cycle, four reduction functions are executed. Between these function calls, a new array of size N is created. By varying N, we can analyze how array size affects the performance of both approaches.

2 Results

All functions were executed 1,000 times to ensure a reliable and consistent comparison. The execution times were measured for different array sizes to observe how performance scales with workload size.

2.1 Discussion

The results show that asynchronous function calls outperform synchronous ones when working with smaller arrays. This is because the CPU and GPU can work in parallel, minimizing idle time and improving throughput.
However, as the array size increases, the GPU becomes the main bottleneck. The time needed for computation and memory transfer dominates, leaving little room for overlap between CPU and GPU tasks. In these cases, the performance advantage of asynchronous execution becomes negligible, and both approaches yield similar total execution times.
In summary, asynchronous GPU calls are more efficient for smaller workloads where overlapping execution is possible, while for larger workloads, the GPU's processing capacity limits performance regardless of the execution mode.

N	Synchronous (ms)	Asynchronous (ms)
32	794.085	746.763
64	782.052	639.148
128	783.046	634.216
256	756.638	678.611
512	1123.55	983.427
1024	1103.71	1060.98
2048	1652.07	1386.09
4096	1879.90	1720.39
8192	2187.84	2033.03
16384	2419.39	2329.57
32768	3415.54	3505.3
65536	5332.03	5302.57
131072	8957.35	8987.88
262144	18110.90	18009.4

Table 1: Comparison of the total program execution time of the synchronous and asynchronous versions as a function of N.

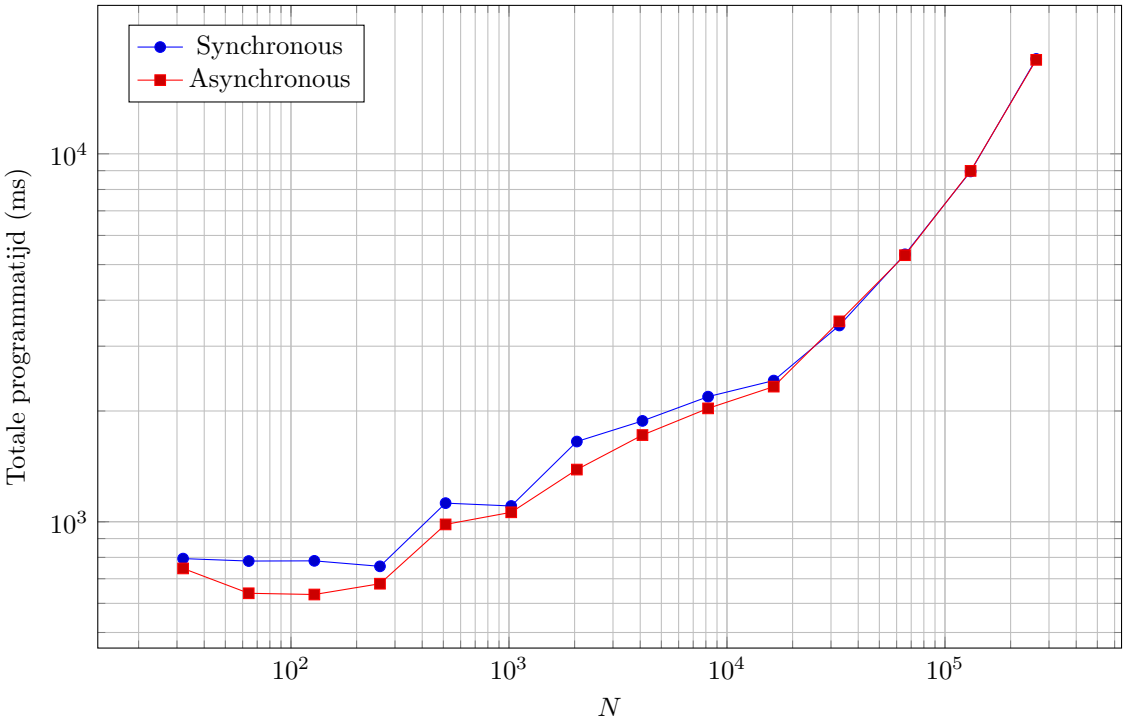


Figure 1: Total program execution time of the synchronous and asynchronous implementation as a function of N (log-log scale).

3 Code

```
int main() {
    clock_t start,end;
    srand(time(NULL));  // seed the random number generator
    for (int N= 32;N < 300000; N*=2){
        int arr[N];
        int numBlocks = 4;
        int blocksPerThread = N / (numBlocks*2);
        start = clock();
        for(int i = 0; i< 1000; i++){
            for (int j = 0; j<4 ; j++){
                for (int d = 0; d < N; d++) {
                    arr[d] = rand() % 1000;
                }
                int Num = 0;
                int *p_arr, *p_Num;
                cudaMalloc((void**)&p_arr, sizeof(int)*N);
                cudaMalloc((void**)&p_Num, sizeof(int));
                cudaMemcpy(p_arr, arr, sizeof(int)*N, cudaMemcpyHostToDevice);
                cudaMemcpy(p_Num, &Num, sizeof(int), cudaMemcpyHostToDevice);
                if (j ==0) reduction_max<<<numBlocks, blocksPerThread>>>(p_arr, p_Num, N);
                else if (j ==1) reduction_min<<<numBlocks, blocksPerThread>>>(p_arr, p_Num, N
                    );
                else if (j ==2) reduction_sum<<<numBlocks, blocksPerThread>>>(p_arr, p_Num, N
                    );
                else if (j ==3) reduction_Sub<<<numBlocks, blocksPerThread>>>(p_arr, p_Num, N
                    );


                cudaDeviceSynchronize () ;
                cudaMemcpy(&Num, p_Num, sizeof(int), cudaMemcpyDeviceToHost);

                cudaFree(p_arr);
                cudaFree(p_Num);

            }
```

```
        }

        cudaDeviceSynchronize();

        end = clock();

        double elapsed_ms = (double)(end - start) * 1000.0 / CLOCKS_PER_SEC;
        std::cout << "Totale programmatijd: " << elapsed_ms << " ms" <<" N: " << N<<   std::
            endl;

    }
    return 0;
}
```

Listing 1: main synchronous

```
int main() {
    clock_t start,end;
    srand(time(NULL));  // seed the random number generator
    for (int N= 32;N < 300000; N*=2){
        int arr[N];
        int numBlocks = 4;
        int blocksPerThread = N / (numBlocks*2);
        start = clock();
        for(int i = 0; i< 1000; i++){
             for (int j = 0; j<4 ; j++){
                for (int d = 0; d < N; d++) {
                    arr[d] = rand() % 1000;
                }
                int Num = 0;
                int *p_arr, *p_Num;
                cudaMalloc((void**)&p_arr, sizeof(int)*N);
                cudaMalloc((void**)&p_Num, sizeof(int));
                cudaMemcpyAsync(p_arr, arr, sizeof(int)*N, cudaMemcpyHostToDevice);
                cudaMemcpyAsync(p_Num, &Num, sizeof(int), cudaMemcpyHostToDevice);
                if (j ==0) reduction_max<<<numBlocks, blocksPerThread>>>(p_arr, p_Num, N);
                else if (j ==1) reduction_min<<<numBlocks, blocksPerThread>>>(p_arr, p_Num, N
                    );
                else if (j ==2) reduction_sum<<<numBlocks, blocksPerThread>>>(p_arr, p_Num, N
                    );
                else if (j ==3) reduction_Sub<<<numBlocks, blocksPerThread>>>(p_arr, p_Num, N
                    );



                cudaMemcpyAsync(&Num, p_Num, sizeof(int), cudaMemcpyDeviceToHost);

                cudaFree(p_arr);
                cudaFree(p_Num);

            }
        }

        cudaDeviceSynchronize();

        end = clock();

        double elapsed_ms = (double)(end - start) * 1000.0 / CLOCKS_PER_SEC;
        std::cout << "Totale programmatijd: " << elapsed_ms << " ms" <<" N: " << N<<   std::
            endl;

    }
    return 0;
}
```

Listing 2: main asynchronous